

FlagGuesseur

Application Android de Quiz Géographique

Rapport Technique Complet

Spécifications Techniques

Plateforme	Android (API 24+)
Langage	Kotlin + Jetpack Compose
Base de données	Room (local) + Supabase (cloud)
Version	1.0
Date	16 mai 2026

Informations Académiques

Étudiant	Wassim Lazim
Encadrant	Mohammed Bousmah
Type	Projet de Fin d'Études
Établissement	École Marocaine des Sciences de l'Ingénieur

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Dédicaces

À ma famille qui m'a soutenu tout au long de ce parcours,

À mes amis qui ont cru en ce projet,

*À mon encadrant Mohammed Bousmah pour ses conseils
précieux et son accompagnement,*

*À tous ceux qui ont contribué de près ou de loin à la réussite
de cette application.*

Wassim Lazim

Remerciements

Je tiens à exprimer mes plus sincères remerciements à :

- **Mohammed Bousmah**, mon encadrant, pour son orientation précieuse, ses conseils avisés et sa disponibilité constante tout au long de ce projet de fin d'études.
- **Les membres du jury** pour le temps consacré à l'évaluation de ce rapport et de l'application.
- **Ma famille et mes proches** pour leur soutien moral et leur encouragement continu.
- **Les contributeurs de la communauté Android** pour les ressources et documentations disponibles en ligne.
- **L'équipe Supabase** pour leur plateforme cloud robuste et leur excellent support.
- **Google Developers** pour les excellentes documentations concernant Android, Kotlin et Jetpack Compose.

Résumé

FlagGuesseur est une application Android native développée en Kotlin avec Jetpack Compose, proposant un quiz interactif de reconnaissance de drapeaux nationaux. L'application couvre plus de 70 pays répartis en 7 catégories géographiques (Europe, Afrique, Asie, Moyen-Orient, Amériques, Océanie et Monde entier).

L'application intègre :

- Un back-end cloud **Supabase** (PostgreSQL) pour la synchronisation des questions et des classements mondiaux
- Une base de données locale **Room** (SQLite) pour le mode hors-ligne
- Un mode duel en temps réel avec polling Supabase
- Une reconnaissance vocale native Android (SpeechRecognizer)
- Un système de gamification complet (couronnes, niveaux, revive)
- 7 modes de jeu variés avec règles et scoring distincts
- Une interface utilisateur moderne utilisant le thème *Atlas Nocturne*

Ce rapport détaille l'architecture MVVM + Repository Pattern, les choix techniques fondamentaux, la structure de données à travers Room et Supabase, les fonctionnalités principales, l'implémentation technique complète, et les perspectives d'évolution du projet.

Mots-clés : Application Android, Kotlin, Jetpack Compose, Quiz géographique, Supabase, Room, MVVM, Gamification, SpeechRecognizer, Duel en temps réel, API REST

Table des matières

Résumé	5
Liste des Abréviations	10
1 Introduction et Contexte Général du Projet	11
1.1 Présentation générale	11
1.2 Motivation du projet	11
2 Architecture et Design	12
2.1 Pattern MVVM + Repository	12
2.2 Stratégie Remote-First avec Fallback	12
3 Fonctionnalités Principales	13
3.1 7 Modes de Jeu	13
3.2 Système de Gamification	13
3.2.1 Couronnes (Monnaie In-App)	13
3.2.2 Niveaux	13
3.2.3 Classement Mondial	13
4 Stack Technologique	15
4.1 Environnement de Développement	15
5 Gestion des Données	16
5.1 Architecture Room (Local)	16
5.2 Supabase (Cloud)	16
6 Détails d'Implémentation	17
6.1 Flow Classique de Jeu	17
6.2 LIA Assistant (OpenRouter)	17
6.3 Mode Duel (Temps Réel)	17
7 Cas d'Usage	18
8 Tests et Validation	19
8.1 Tests Unitaires	19
8.2 Tests d'Intégration	19
8.3 Tests UI	19
9 Déploiement	20

9.1	Build Release	20
9.2	Configuration Overleaf	20
10	Perspectives Futures	21
10.1	Court Terme	21
10.2	Moyen Terme	21
10.3	Long Terme	21
A	Diagrammes UML	22
1.1	Use Cases	22
B	Code Source Clés	23
2.1	Structure du Projet	23
C	Ressources et Références	24

Table des figures

Liste des tableaux

Liste des Abréviations

Abréviation	Signification
API	Application Programming Interface
AVD	Android Virtual Device
CSV	Comma-Separated Values
DAO	Data Access Object
FCM	Firebase Cloud Messaging
FPS	Frames Per Second
Git	Version Control System
GSON	Google Serialization/Deserialization
HTTP/HTTPS	HyperText Transfer Protocol / Secure
IDE	Integrated Development Environment
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
KSP	Kotlin Symbol Processing
Material Design	Design Language de Google
MVVM	Model-View-ViewModel
OAuth	Open Authorization
OkHttp	HTTP Client Library
PostgreSQL	Système de gestion de base de données
REST	Representational State Transfer
RLS	Row Level Security
SDK	Software Development Kit
SharedPreferences	Stockage de préférences Android
SpeechRecognizer	API Android de reconnaissance vocale
SQLite	Base de données embarquée
StateFlow	Flow coroutine pour états réactifs
Supabase	Backend-as-a-Service Open Source
UI	User Interface
UUID	Universally Unique Identifier
ViewModel	Classe Android gestion d'état UI

Chapitre 1

Introduction et Contexte Général du Projet

1.1 Présentation générale

FlagGuesseur est une application mobile conçue pour rendre l'apprentissage de la géographie ludique et engageant. Basée sur la reconnaissance des drapeaux nationaux, elle propose une expérience interactive multimodale combinant quiz traditionnel, reconnaissance vocale, et compétition en temps réel.

1.2 Motivation du projet

Le projet répond à plusieurs objectifs :

- Créer une application pédagogique accessible et divertissante
- Démontrer la maîtrise des technologies modernes Android (Kotlin, Jetpack Compose)
- Implémenter une architecture évolutive (MVVM, Repository Pattern)
- Intégrer des services cloud avec stratégie de synchronisation robuste
- Fournir une expérience utilisateur fluide sur 60 FPS constant

Chapitre 2

Architecture et Design

2.1 Pattern MVVM + Repository

L'architecture suit le pattern Model-View-ViewModel couplé à Repository Pattern :

Architecture globale UI (Compose) → ViewModel → Repository → Data Sources
(Room/Supabase/Seed)

2.2 Stratégie Remote-First avec Fallback

- 0. **Tentative Supabase** : Récupération des questions et leaderboard frais
- 0. **Succès** : Cache dans Room + Retour au UI
- 0. **Erreur réseau** : Basculer automatiquement sur le cache Room
- 0. **Cache vide** : Utiliser les SeedData embarquées

Chapitre 3

Fonctionnalités Principales

3.1 7 Modes de Jeu

- 0. **Classique** : Mode standard (3 vies, 15s/question, points standard)
- 0. **Survie** : Timer accéléré (1 vie, points doublés, difficulté progressive)
- 0. **Contre-la-montre** : 60 secondes partagées pour toutes les questions
- 0. **Capitales** : Identifier la capitale du pays montré
- 0. **Zen** : Apprentissage sans stress (pas de timer, pas de vies)
- 0. **Vocal** : Prononciation du nom du pays via SpeechRecognizer
- 0. **Duel** : 1v1 en temps réel avec synchronisation Supabase (polling 2s)

3.2 Système de Gamification

3.2.1 Couronnes (Monnaie In-App)

- Indice déverrouille : 50 couronnes
- Revive après défaite : 200-1000 couronnes (basé sur difficulté)

3.2.2 Niveaux

L'application propose 6 niveaux progressifs basés sur couronnes accumulées :

- 0. Novice
- 0. Apprenti
- 0. Explorateur
- 0. Aventurier
- 0. Expert
- 0. Maître des Nations

3.2.3 Classement Mondial

Top 100 en temps réel avec :

- Synchronisation Supabase automatique

- Notifications au nouveau record personnel
- Vue globale et statistiques détaillées

Chapitre 4

Stack Technologique

4.1 Environnement de Développement

Catégorie	Technologie
Langage	Kotlin 2.x
UI Framework	Jetpack Compose + Material Design 3
Architecture Pattern	MVVM + Repository
Base de données locale	Room 2.x (SQLite)
Base de données cloud	Supabase (PostgreSQL)
Réseau	Retrofit 2 + OkHttp 4
Gestion d'images	Coil 2 (SVG support)
Reconnaissance vocale	Android SpeechRecognizer
Animations	Lottie + Compose Animations
API Externe	OpenRouter (LIA assistant)
Minimum SDK	API 24 (Android 7.0)
Target SDK	API 35 (Android 15)

Chapitre 5

Gestion des Données

5.1 Architecture Room (Local)

Room gère :

- Cache des questions récupérées depuis Supabase
- Solde de couronnes utilisateur
- Statistiques et scores locaux
- Leaderboard local (fallback)

5.2 Supabase (Cloud)

Supabase fournit :

- Table principale : `questions` (pays, catégories, options, drapeaux)
- Table leaderboard : `leaderboard` (utilisateur, score, couronnes)
- Table duel en temps réel : `duel_rooms`
- Row-Level Security (RLS) pour sécurité

Chapitre 6

Détails d'Implémentation

6.1 Flow Classique de Jeu

- 0. Utilisateur sélectionne mode + catégorie
- 0. Repository tente Supabase → fallback Room
- 0. Questions affichées avec drapeau (Image Coil SVG)
- 0. Utilisateur répond parmi 4 options
- 0. Score calculé (bonus temps restant en mode Classique)
- 0. Couronnes ajoutées au solde Room
- 0. Leaderboard synchronisé vers Supabase

6.2 LIA Assistant (OpenRouter)

Assistant éducatif Powered by OpenRouter API avec fallback sur données prédéfinies pour questions QCM par pays.

Fonctionnalités LIA :

- Explications automatiques pour chaque réponse correcte
- Chat libre sur géographie et culture générale
- Génération QCM sur sujet spécifique

6.3 Mode Duel (Temps Réel)

Implémentation :

- 0. Appairage via Supabase query (recherche joueur disponible)
- 0. Synchronisation score toutes les 2 secondes (polling)
- 0. Vainqueur déclaré quand premier franchit threshold (ex : 10 bonnes réponses)
- 0. Résultat sauvegardé dans `duel_history`

Chapitre 7

Cas d'Usage

L'application propose 5 cas d'usage principaux détaillés en annexe UML :

- 0. Jouer en mode Classique (et variantes)
- 0. Affronter un adversaire en Duel
- 0. Débloquer Indice (consommer couronnes)
- 0. Consulter Classement mondial
- 0. Utiliser Mode Vocal

Voir diagramme des cas d'usage : `usecases.uml`

Chapitre 8

Tests et Validation

8.1 Tests Unitaires

À implémenter : Tests pour calculateur de score, validateur de réponses, etc.

8.2 Tests d'Intégration

À implémenter : Tests Room, Supabase synchronization, OpenRouter integration.

8.3 Tests UI

À implémenter : Tests Compose pour écrans principaux, navigation, état.

Chapitre 9

Déploiement

9.1 Build

Release

```
1 ./gradlew bundleRelease # Google Play App Bundle
2 ./gradlew build         # APK Debug/Release
```

9.2 Configuration

Overleaf

Sur Overleaf (si compilé à partir du LaTeX) :

- **Compilateur requis** : XeLaTeX (pour texte arabe)
- Menu : Overleaf > Menu > Compiler > XeLaTeX
- Tous les fichiers images doivent être dans le dossier **assets/**

Chapitre 10

Perspectives Futures

10.1 Court

Terme

- Implémenter tests automatisés (unit + integration)
- Optimiser requêtes Supabase (indexation)
- Analytics pour suivi utilisateurs

10.2 Moyen

Terme

- Système d'amis et invitations directes pour duel
- Tournois hebdomadaires avec récompenses
- Modes multijoueur (4+ joueurs)
- Support hors-ligne complète (mode Zen sans réseau)

10.3 Long

Terme

- Extension iOS via Kotlin Multiplatform
- Backend custom (remplacer OpenRouter par fine-tuned LLM interne)
- Intégration réseaux sociaux (partage scores)
- Monétisation (cosmétiques, pass premium)

Annexe A

Diagrammes UML

1.1 Use

Cases

Voir fichier séparé : `usecases.uml`

Les diagrammes suivants doivent être inclus :

- Architecture MVVM
- Flux synchronisation Supabase
- Flux duel en temps réel

Annexe B

Code Source Clés

2.1 Structure du Projet

```
1  app/src/main/
2  java/com/example/quiz_app_wvssim/
3  data/
4      local/          (Room DAO, Database)
5      remote/         (Retrofit, Supabase API)
6      repository/     (Repository Pattern)
7  domain/            (Entities, Use Cases)
8  ui/
9      screen/         (Composables écrans)
10     components/     (Composables réutilisables)
11     viewmodel/      (ViewModels)
12     MainActivity.kt
13 resources/
14     drawable/        (Assets, icônes)
```


Annexe C

Ressources et Références

- 0. Android Official Documentation : <https://developer.android.com>
- 0. Jetpack Compose Guide : <https://developer.android.com/jetpack/compose>
- 0. Room Database : <https://developer.android.com/training/data-storage/room>
- 0. Supabase Documentation : <https://supabase.com/docs>
- 0. Kotlin Documentation : <https://kotlinlang.org/docs>